

The Conjugate Gradient Method for Solving Sparse Symmetric Positive Definite Linear Systems

Jacek Schmidt, jacek14schmidt@gmail.com

August 2025

1 Introduction

Linear systems of the form

$$A\mathbf{x} = \mathbf{b}$$

arise frequently across applied mathematics as a means of describing the storage, interaction, and transformation of information. They arise in a wide range of applications: in engineering and physics, they approximate partial differential equations (PDEs) governing structural mechanics, fluid flow, heat conduction, diffusion, and gravitational or electromagnetic fields; in data science, they appear in least-squares parameter estimation, ridge regression, and Gaussian processes; in finance, they are used for portfolio optimisation and risk management; and they also find roles in computer graphics and network analysis.

Given their broad importance, numerous methods have been developed to solve linear systems. The most suitable method depends on the dimension of the problem, the condition number of the matrix, and the matrix's structural properties.

Direct methods work by factorising the matrix A , for example:

$$A = LU, \quad PA = LU \quad (\text{with pivoting}), \quad \text{or} \quad A = LL^\top \text{ if } A \text{ is symmetric positive definite (SPD).}$$

These factorisations make the resulting system significantly easier to solve. LU factorisation with pivoting, a reordering of the equations, works for every non-singular matrix and is therefore considered highly robust. However, its computational cost is significant, with complexity on the order of $\mathcal{O}(n^3)$, making it unsuitable for very large systems.

Iterative methods instead begin with an initial guess and progressively refine it. Among these, the conjugate gradient method (CGM) is widely used for SPD matrices. It guarantees convergence within at most n iterations (where n is the dimension of the system) but often converges much sooner. The convergence rate depends largely on the matrix condition number: matrices with large condition numbers lead to slower convergence. Preconditioning is often employed to reduce the condition number while preserving the exact solution, although the suitability (in terms of quality and cost) of each preconditioner varies by problem.

The CGM is particularly effective for sparse SPD matrices, which frequently arise from PDE discretisations. Sparse matrices significantly reduce the computational cost of the key matrix-vector product $A\mathbf{p}_k$ within each iteration. While non-SPD systems can often be transformed into SPD ones (for example, via the normal equations), this transformation comes at the cost of its own implementation as well as squaring the condition number, often increasing the demand for high-quality, costly preconditioners.

Other iterative methods include GMRES, which is applicable to general non-singular (and some singular) matrices, including complex systems. However, since GMRES does not exploit the SPD

property, the CGM is generally preferred when SPD applies.

Specialised techniques such as multigrid methods also exist. Multigrid approaches solve the problem on gradually increasing scales which allows it to identify and remove errors effectively. While powerful, they are complex and computationally costly to implement, particularly when geometric or algebraic multigrid (AMG) setups are needed. They are most effective for large-scale two- and three-dimensional PDEs, especially where condition numbers exceed approximately 10^6 , making standard preconditioners with iterative methods impractical.

This project focuses on the formulation of the conjugate gradient method and demonstrates its implementation through practical examples. Additionally, it discusses how preconditioning can be used to improve convergence by reducing the condition number, thereby maintaining the method's efficiency for challenging problems.

2 CGM Formulation

We begin with the linear system

$$A\mathbf{x} = \mathbf{b},$$

which arises frequently in applied Mathematics. One immediate idea is to multiply by the inverse of A . Directly forming A^{-1} , however, is rarely practical and standard direct algorithms such as Gaussian elimination or LU decomposition require on the order of $\mathcal{O}(n^3)$ operations, and even with sparsity this approach becomes prohibitively expensive for large-scale systems.

A more natural first step is to recast the linear system as a minimisation problem. Consider the residual

$$\mathbf{r} = A\mathbf{x} - \mathbf{b},$$

and the associated functional

$$\hat{f}(\mathbf{x}) = \mathbf{r}^\top \mathbf{r} = (A\mathbf{x} - \mathbf{b})^\top (A\mathbf{x} - \mathbf{b}),$$

which measures the squared L_2 norm of the residual. Minimising $\hat{f}(\mathbf{x})$ is equivalent to solving $A\mathbf{x} = \mathbf{b}$, since $\hat{f}(\mathbf{x}) = 0$ if and only if the residual vanishes. A simple iterative strategy is to choose an initial guess $\mathbf{x}_0$ and generate a sequence $\{\mathbf{x}_k\}$ such that

$$\|A\mathbf{x}_{k+1} - \mathbf{b}\|_2 \leq \|A\mathbf{x}_k - \mathbf{b}\|_2.$$

The gradient of $\hat{f}$ is

$$\nabla \hat{f}(\mathbf{x}) = 2A^\top (A\mathbf{x} - \mathbf{b}),$$

and using this gradient in a steepest descent scheme does lead to convergence when A is invertible. However, two important limitations appear. First, for ill-conditioned systems (ones with sensitive solutions which lead to slow convergences) or poor initial guesses, gradient descent may require a large number of iterations. Second, evaluating $\nabla \hat{f}(\mathbf{x})$ involves two matrix-vector multiplications per step, which makes each iteration computationally expensive.

A straightforward improvement is to change the functional so that its gradient is simply the residual itself. Define

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top A \mathbf{x} - \mathbf{b}^\top \mathbf{x}, \quad \nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}.$$

This formulation assumes that A is symmetric, because only then does the gradient correspond to the residual of the original system. This choice preserves the property that stationary points of f coincide with solutions of the linear system, but now the gradient requires only a single matrix-vector multiplication. This resolves one of the bottlenecks of the naive $\hat{f}$ formulation, although for challenging problems the method can still converge slowly if the descent directions are not chosen effectively.

To motivate the conjugate gradient method, it is useful to think in terms of building the solution from a set of basis vectors. Let $\{\mathbf{e}_i\}$ be a basis of $\mathbb{R}^n$ (not necessarily orthogonal or normalised), and let $\mathbf{x}_e$ be the exact solution of $A\mathbf{x} = \mathbf{b}$. We may write

$$\mathbf{x}_e = \sum_i \phi_i \mathbf{e}_i.$$

Multiplying the system by $\mathbf{e}_j^\top$ gives

$$\mathbf{e}_j^\top \mathbf{b} = \mathbf{e}_j^\top A \mathbf{x}_e = \sum_i \phi_i \mathbf{e}_j^\top A \mathbf{e}_i.$$

If the basis vectors are chosen to be *conjugate* with respect to A , meaning

$$\mathbf{e}_j^\top A \mathbf{e}_i = 0 \quad \text{for } i \neq j,$$

then this simplifies to

$$\mathbf{e}_j^\top \mathbf{b} = \phi_j \mathbf{e}_j^\top A \mathbf{e}_j,$$

and the coefficients follow directly:

$$\phi_j = \frac{\mathbf{e}_j^\top \mathbf{b}}{\mathbf{e}_j^\top A \mathbf{e}_j}. \quad (1)$$

This procedure would reconstruct $\mathbf{x}_e$ exactly in at most n iterations (ignoring round-off). Each iteration involves only a single matrix–vector multiplication, so the per-step cost is modest. However, the quality of intermediate guesses depends strongly on the choice and ordering of the basis vectors. Without a good sequence of conjugate directions, the approximation may remain poor until all n components are included.

The conjugate gradient method combines the efficiency of using the residual as a gradient with the idea of iteratively building the solution in a set of A -conjugate directions. At each step, the new search direction is chosen to be A -orthogonal (conjugate) to the previous ones, and the step length is selected to minimise the quadratic functional along that direction. As a result, CG converges in at most n steps in exact arithmetic, but in practice often reaches a sufficiently accurate solution in far fewer iterations. Each iteration requires only a few vector operations and a single multiplication by A , making the method attractive for large sparse systems where forming or storing A^{-1} is infeasible.

2.1 Step Size Selection

We recall the quadratic functional associated with our linear system:

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top A \mathbf{x} - \mathbf{b}^\top \mathbf{x}, \quad \nabla f(\mathbf{x}) = A \mathbf{x} - \mathbf{b},$$

where A is assumed to be symmetric. We seek to construct an iterative process

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k,$$

where $\mathbf{p}_k$ is a chosen search direction and α_k is a scalar step size to be determined. Our aim is to select α_k such that the value of $f(\mathbf{x})$ is minimised along the line defined by $\mathbf{x}_k + \alpha \mathbf{p}_k$. An illustration of this line search is given in Figure 2.

Substituting this update into the functional,

$$f(\mathbf{x}_k + \alpha \mathbf{p}_k) = \frac{1}{2} (\mathbf{x}_k + \alpha \mathbf{p}_k)^\top A (\mathbf{x}_k + \alpha \mathbf{p}_k) - \mathbf{b}^\top (\mathbf{x}_k + \alpha \mathbf{p}_k),$$

we expand and group the terms:

$$f(\mathbf{x}_k + \alpha \mathbf{p}_k) = f(\mathbf{x}_k) + \frac{\alpha^2}{2} \mathbf{p}_k^\top A \mathbf{p}_k + \alpha \mathbf{p}_k^\top (A \mathbf{x}_k - \mathbf{b}).$$

The optimal step size corresponds to a stationary point of $f(\mathbf{x}_k + \alpha \mathbf{p}_k)$ with respect to α . Differentiating with respect to α and setting the result to zero,

$$\frac{\partial f}{\partial \alpha} = \alpha \mathbf{p}_k^\top A \mathbf{p}_k + \mathbf{p}_k^\top (A \mathbf{x}_k - \mathbf{b}) = 0,$$

we obtain

$$\alpha_k = \frac{\mathbf{p}_k^\top (\mathbf{b} - A \mathbf{x}_k)}{\mathbf{p}_k^\top A \mathbf{p}_k}. \quad (2)$$

Substituting this optimal α_k back into the expression for $f(\mathbf{x}_{k+1})$ yields

$$f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k) - \frac{1}{2} \frac{[\mathbf{p}_k^\top (\mathbf{b} - A \mathbf{x}_k)]^2}{\mathbf{p}_k^\top A \mathbf{p}_k}.$$

This formula shows that the reduction in f is always non-positive provided the denominator is positive.

If A is *positive definite*, that is,

$$\mathbf{x}^\top A \mathbf{x} > 0 \quad \text{for all } \mathbf{x} \neq \mathbf{0},$$

then $\mathbf{p}_k^\top A \mathbf{p}_k > 0$ for any nonzero $\mathbf{p}_k$, and the above step size guarantees that

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k).$$

Since a positive definite matrix is also invertible, the functional $f(\mathbf{x})$ has a unique minimiser, and this line-search procedure ensures convergence to that minimiser provided suitable search directions are chosen.

2.2 Direction Vector Selection

Having determined how to choose an optimal step size, we now return to the problem of selecting a search direction.

One possibility is to choose an arbitrary set of A -conjugate basis vectors $\{\mathbf{e}_i\}$ and to use them sequentially as search directions. Although this approach may initially seem artificial, it has a useful property:

$$\mathbf{e}_j^\top A \mathbf{x}_k = \mathbf{e}_j^\top \mathbf{b} \quad \text{for all } j < k.$$

The proof of this result is given in the Theorem 8.1 within Appendix 8.1. Its interpretation is that each iteration exactly corrects one additional component of the system while preserving all previously corrected components. As a consequence, the process converges exactly in n steps (up to numerical error). Figure 2(a) and 2(c) illustrate this behaviour for two different choices of conjugate directions.

However, this approach has a notable drawback: although convergence is guaranteed in n iterations, the intermediate approximations may remain poor until the final steps. In Figure 2(a), for instance, the first update provides little improvement, and the majority of the correction occurs in the second iteration. Ideally, the largest improvements should occur early, so that only a few iterations are needed to achieve sufficient accuracy.

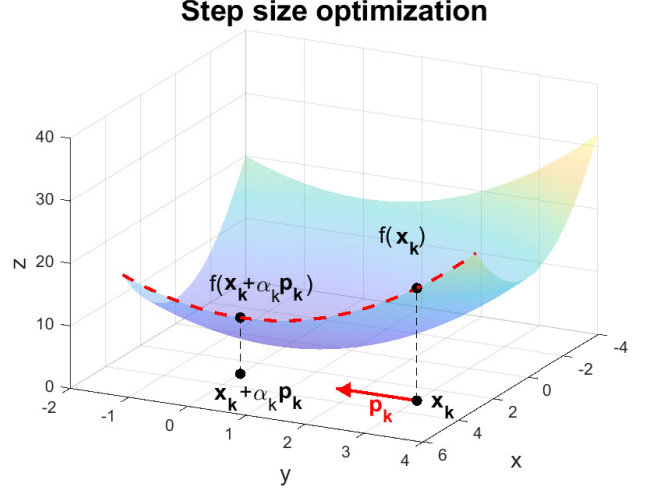


Figure 1: Illustration of optimal step size selection. Here, for the simple function $z = x^2 + y^2$, starting at $(3, 3)$ and moving along the direction $(-1, 0)$, the optimal step size is $\alpha = 3$, leading directly to the minimum in the x -direction.

A different strategy is to use the gradient of the functional as the search direction at each iteration. This leads to the classical steepest gradient descent method. Its main advantage is that the initial iterations usually make large corrections to the guess, bringing it significantly closer to the true solution. This is illustrated in Figure 2(b), where the first step accounts for most of the correction. The disadvantage, however, is that later iterations often contribute very little improvement, and in practice this method frequently requires more than n iterations to reach the desired tolerance.

Ideally, we would like a method that combines the benefits of both approaches: guaranteed convergence in at most n iterations, while also ensuring that the early iterations provide the largest improvements. This can be achieved by constructing a sequence of search directions that are both conjugate to one another and closely aligned with the steepest descent directions.

One such construction starts from the residuals $\mathbf{r}_i$ (the negative gradients of the functional—which coincides with the direction of steepest descent) and adjusts them to enforce mutual A -conjugacy:

$$\mathbf{p}_i = \mathbf{r}_i - \sum_{j < i} \frac{\mathbf{p}_j^\top A \mathbf{r}_i}{\mathbf{p}_j^\top A \mathbf{p}_j} \mathbf{p}_j. \quad (3)$$

The proof that this recurrence produces a set of mutually conjugate vectors is given in Theorem 2 within Appendix 8.2. Intuitively, each search direction starts from the steepest descent direction $\mathbf{r}_i$, with components that violate conjugacy removed by projection onto the previously constructed directions.

This approach forms the basis of the *conjugate gradient method*, which inherits the rapid initial progress of steepest descent together with the finite-step convergence property of conjugate direction methods.

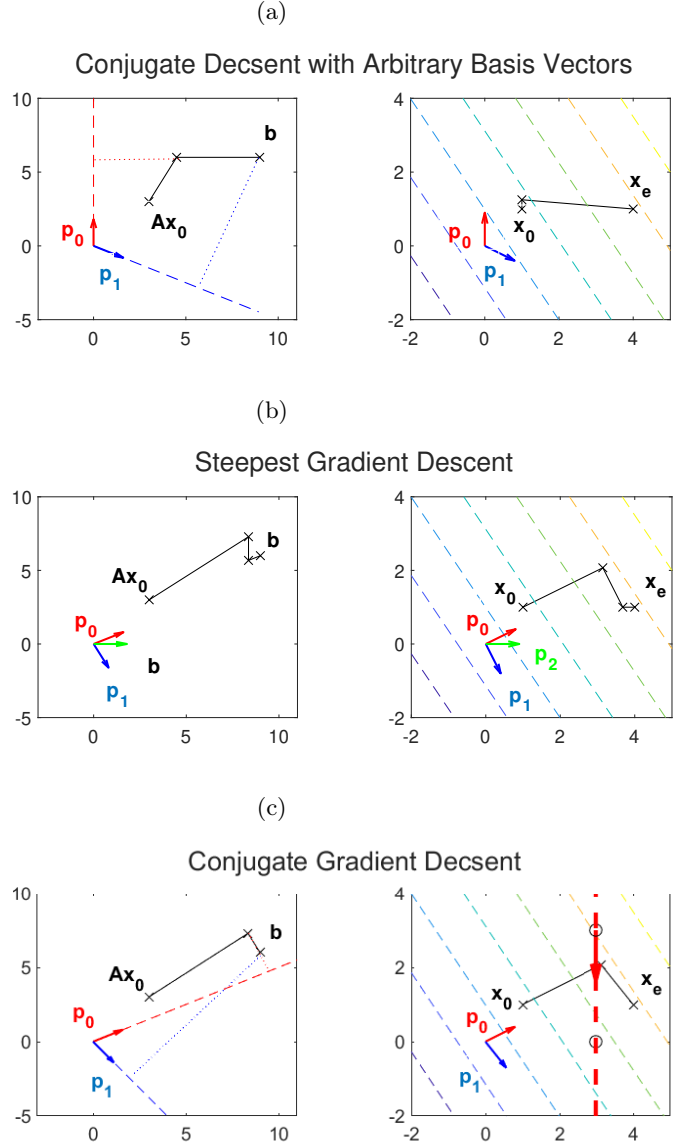


Figure 2: The iterations using different search directions. (a) uses arbitrary basis vectors $(0, 1)^\top$ and $(2, -1)^\top$, (b) uses the gradient function and (c) uses the conjugate gradient function. The linear system in question is $\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \mathbf{x} =$

$$\begin{pmatrix} 9 \\ 6 \end{pmatrix} \text{ where } \mathbf{x}_0 = (1, 1)^\top.$$

2.3 Practical Formulation

The iterative procedure introduced so far can be written as

$$\begin{aligned}
\mathbf{r}_k &= \mathbf{b} - A\mathbf{x}_k, \\
\mathbf{p}_k &= \mathbf{r}_k - \sum_{j < k} \frac{\mathbf{p}_j^\top A \mathbf{r}_k}{\mathbf{p}_j^\top A \mathbf{p}_j} \mathbf{p}_j, \\
\alpha_k &= \frac{\mathbf{p}_k^\top \mathbf{r}_k}{\mathbf{p}_k^\top A \mathbf{p}_k}, \\
\mathbf{x}_{k+1} &= \mathbf{x}_k + \alpha_k \mathbf{p}_k.
\end{aligned} \tag{4}$$

In this form, however, each iteration is computationally expensive. Both the residual $\mathbf{r}_k$ and the search direction $\mathbf{p}_k$ require several matrix–vector multiplications, making the algorithm inefficient for large systems. A more practical formulation can be obtained by expressing key quantities recursively, so that each step reuses previously computed information.

We begin with the residual. Starting from its definition,

$$\mathbf{r}_k = \mathbf{b} - A\mathbf{x}_k, \quad \mathbf{r}_{k-1} = \mathbf{b} - A\mathbf{x}_{k-1},$$

their difference is

$$\mathbf{r}_k - \mathbf{r}_{k-1} = A(\mathbf{x}_{k-1} - \mathbf{x}_k) = -\alpha_{k-1} A\mathbf{p}_{k-1},$$

and thus

$$\mathbf{r}_k = \mathbf{r}_{k-1} - \alpha_{k-1} A\mathbf{p}_{k-1}. \tag{5}$$

Next, the Gram–Schmidt process for constructing mutually conjugate directions can be written in a simple recurrence form (see Theorem 6 in the Appendix 8.6 for the proof):

$$\mathbf{p}_k = \mathbf{r}_k + \beta_{k-1} \mathbf{p}_{k-1}, \quad \beta_{k-1} = \frac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{r}_{k-1}^\top \mathbf{r}_{k-1}}. \tag{6}$$

This is a significant improvement: it avoids a full Gram–Schmidt sum over all previous directions and replaces it with a short recurrence that involves only the most recent pair of residual and direction vectors.

Finally, Theorem 3 (see Appendix 8.3) states that $\mathbf{p}_k^\top \mathbf{r}_k = \mathbf{r}_k^\top \mathbf{r}_k$, allowing the step size to be expressed as

$$\alpha_k = \frac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{p}_k^\top A \mathbf{p}_k}. \tag{7}$$

Although this does not reduce the number of matrix–vector products directly, it is numerically more stable, because $\mathbf{r}_k^\top \mathbf{r}_k$ is already computed for the convergence criterion, and its use avoids the propagation of rounding errors that could arise from $\mathbf{p}_k^\top \mathbf{r}_k$.

Combining equation (5), (6) and (7), we arrive at the commonly used practical formulation of the conjugate gradient method:

$$\begin{aligned}
&\mathbf{x}_0 \text{ given,} \\
&\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0, \\
&\mathbf{p}_0 = \mathbf{r}_0, \\
&\text{for } k \geq 0 : \\
&\quad \alpha_k = \frac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{p}_k^\top A \mathbf{p}_k}, \\
&\quad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k, \\
&\quad \mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A\mathbf{p}_k, \\
&\quad \beta_k = \frac{\mathbf{r}_{k+1}^\top \mathbf{r}_{k+1}}{\mathbf{r}_k^\top \mathbf{r}_k}, \\
&\quad \mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k.
\end{aligned} \tag{8}$$

This reformulation reduces each iteration to a single matrix–vector multiplication, $\mathbf{A}\mathbf{p}_k$, which dominates the computational cost.

3 One-Dimensional Poisson Problem Example

The one-dimensional Poisson equation is written as

$$-u''(x) = f(x), \quad a \leq x \leq b, \quad (9)$$

subject to suitable boundary conditions, such as Dirichlet, Neumann, or mixed. This equation arises in many physical contexts: in steady heat conduction, $u(x)$ represents the temperature profile and $f(x)$ denotes an internal heat generation term; in electrostatics, $u(x)$ is the electric potential and $f(x)$ corresponds to charge density; in structural mechanics, $u(x)$ may represent the deflection of a beam and $f(x)$ the distributed load along it. For the one-dimensional setting, a common choice is homogeneous Dirichlet conditions, $u(a) = u(b) = 0$, which correspond to fixed temperature at both ends or a beam clamped at its boundaries.

To solve this equation numerically, the interval $[a, b]$ is divided into n interior points, with step size

$$h = \frac{b - a}{n + 1}.$$

Let u_i denote the numerical approximation to $u(x)$ at $x = a + ih$ for $i = 1, \dots, n$. A central finite difference approximation for the second derivative is

$$u''(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2}.$$

Substituting this into the equation (9) gives

$$\begin{aligned} -\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} &= f_i \\ \implies -u_{i+1} + 2u_i - u_{i-1} &= h^2 f_i, \end{aligned}$$

where $f_i = f(x_i)$. Collecting these equations for $i = 1, \dots, n$ leads to the standard tridiagonal linear system

$$\begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ 0 & -1 & 2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & -1 \\ 0 & 0 & 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_n \end{pmatrix} = h^2 \begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ \vdots \\ f_n \end{pmatrix}. \quad (10)$$

For the special case $f(x) \equiv 1$, the right-hand side becomes $h^2 [1, \dots, 1]^\top$. Physically, a constant f represents a uniform heat source or a constant distributed load.

Before we dive into using CGM to solve this linear system, we need to check that the matrix is both symmetric and positive definite. It's symmetry is self evident so we move onto its positive definiteness. In general, for an n dimensional problem, $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ is given by $\sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j$ where a_{ij} is the i - j th entry of $\mathbf{A}$. For a symmetric matrix this can be rewritten as

$$\sum_{i=1}^n a_{ii} x_i^2 + 2 \sum_{1 \leq i < j \leq n} a_{ij} x_i x_j.$$

Using $a_{ii} = 2$ and $a_{i,i+1} = a_{i+1,i} = -1$,

$$\mathbf{x}^\top \mathbf{A} \mathbf{x} = \sum_{i=1}^n 2x_i^2 + 2 \sum_{1 \leq i < j \leq n} a_{ij} x_i x_j = \sum_{i=1}^n 2x_i^2 + 2 \sum_{i=1}^n (-1) x_i x_{i+1} = 2 \sum_{i=1}^n x_i^2 - 2 \sum_{i=1}^n x_i x_{i+1}.$$

This can be rewritten as

$$\mathbf{x}^\top A \mathbf{x} = \sum_{i=1}^{n-1} (x_i - x_{i+1})^2 + x_1^2 + x_n^2,$$

which is greater than 0 for all non-zero $\mathbf{x}$.

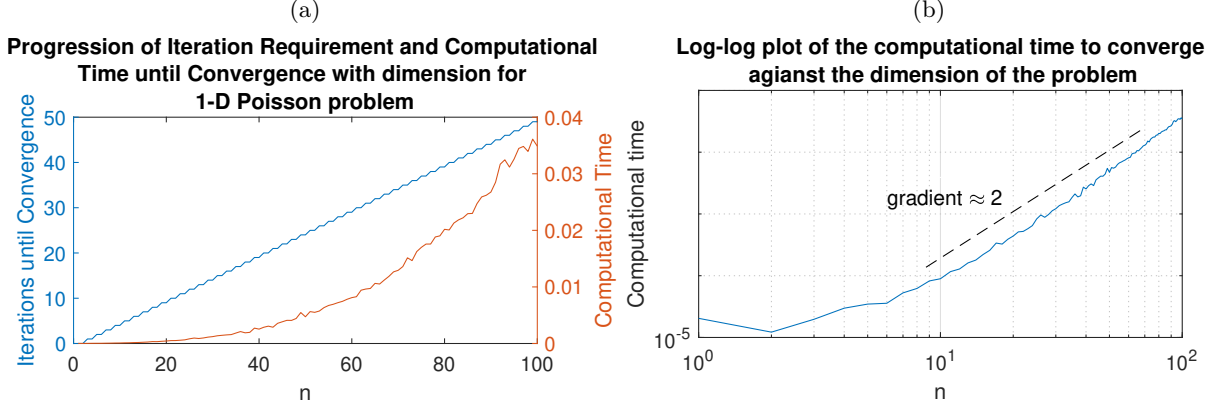


Figure 3: (a) Number of iterations and computational time required to solve system (9) with the CGM as a function of the number of interior points n . (b) Log-log plot of the computational time to achieve a given tolerance against n . The data suggest a growth in computational cost that approaches quadratic in n .

Now that we have verified that the matrix is symmetric and positive definite we can use the CGM to solve the linear system. Figure 3(a) shows how the number of iterations required for convergence, and the corresponding computational time, scale as the number of interior points increases. Each iteration is dominated by a sparse matrix-vector product, which for a tridiagonal matrix has complexity $\mathcal{O}(n)$. Since the number of iterations for the one-dimensional Poisson operator tends to grow proportionally to n , the total computational time scales close to $\mathcal{O}(n^2)$, consistent with the trend observed in Figure 3(b).

System (9) admits an analytical solution in the case of constant forcing, $f(x) \equiv 1$. Solving

$$u_{i+1} - 2u_i + u_{i-1} = -h^2, \quad u_0 = u_{n+1} = 0,$$

yields

$$u_i = a + \frac{(b-a)h^2}{2} i(n+1-i), \quad i = 1, \dots, n. \quad (11)$$

Introducing the scaled spatial variable $s_i = a + ih$ for the interval $[a, b]$, this can be expressed as

$$u(s) = \frac{1}{2} (s-a)(b-s), \quad (12)$$

showing that the shape of the solution is independent of the grid resolution; refining the mesh simply samples this parabola more finely. Furthermore, the solution in equation (12) matches the analytic solution to system (9) which can easily be verified to be quadratic

Convergence of $\mathbf{x}_k$ for increasing iterations

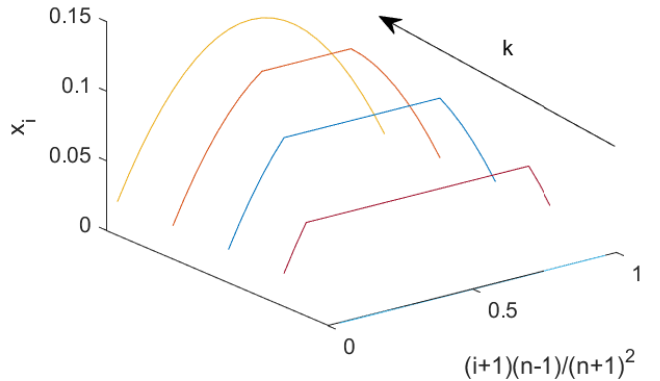


Figure 4: Convergence of the CGM's solution of system (9) towards the analytical solution (12) over successive iterations. Here, $a = 0$ and $b = 1$ for illustrative purposes.

with -0.5 as the leading coefficient and roots of a and b .

Figure 4 illustrates the convergence of the CGM towards the exact solution as the number of iterations increases, validating the correctness of the implementation.

4 Two-Dimensional Poisson Problem Example

In the one-dimensional Poisson problem, we examined how a variable evolved with respect to a single spatial coordinate, for instance, the deflection of a beam along its length or the temperature distribution along a rod. Extending this to two dimensions, the Poisson equation becomes

$$\nabla^2 u(x, y) = 1, \quad (13)$$

where $(x, y) \in [a_1, a_2] \times [b_1, b_2]$ and subject to homogeneous Dirichlet boundary conditions,

$$u(x, a_2) = u(x, b_2) = u(a_1, y) = u(b_1, y) = 0.$$

This formulation can model, for example, a thin square plate subjected to a uniform load with its edges clamped. Alternatively, if the second variable is temporal, it may represent the progression of a heat distribution within a rod where the distribution is constant at both the start and end and where the ends of the rod are kept at constant temperature.

After scaling the domain so that each length is equal, we can discretise it by dividing each spatial direction into n internal points, giving a total of n^2 internal nodes. Let the internal grid points be

$$(x_i, y_j) = (a_1 + ih, b_1 + jh), \quad \text{for } i, j = 1, \dots, n \quad \text{where } h = \frac{b_1 - a_1}{n + 1},$$

is the uniform step size. Using a standard five-point stencil finite difference approximation for the Laplacian, the resulting discrete system can be expressed in block matrix form as

$$\begin{pmatrix} B & -I & 0 & \cdots & 0 \\ -I & B & -I & \cdots & 0 \\ 0 & -I & B & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & -I \\ 0 & 0 & 0 & -I & B \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{n^2} \end{pmatrix} = h^2 \begin{pmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}, \quad (14)$$

where each block is an $n \times n$ matrix and

$$B = \begin{pmatrix} 4 & -1 & 0 & \cdots & 0 \\ -1 & 4 & -1 & \cdots & 0 \\ 0 & -1 & 4 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & -1 \\ 0 & 0 & 0 & -1 & 4 \end{pmatrix}.$$

The global matrix A in this system is symmetric by construction, and one can verify that

$$\mathbf{x}^\top A \mathbf{x} = \sum_{i,j} [(x_{i,j} - x_{i+1,j})^2 + (x_{i,j} - x_{i,j+1})^2] > 0 \quad \text{for } \mathbf{x} \neq \mathbf{0},$$

which confirms that A is also positive definite. Hence, the CGM can be applied to solve system (14). The solution for a representative case is shown in Figure 5.

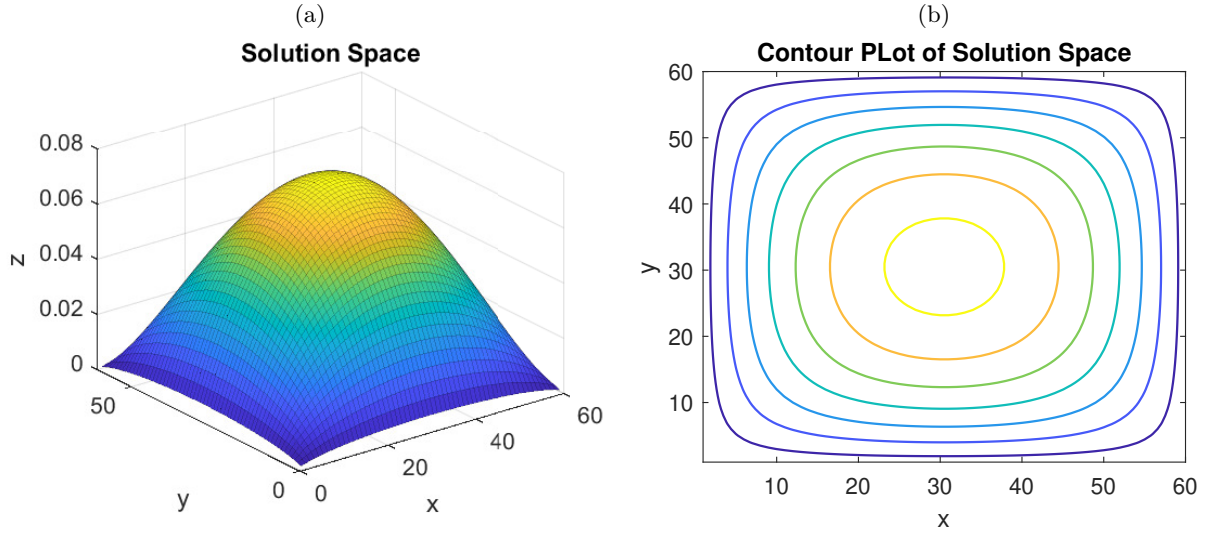


Figure 5: Solution to equation (14) using the CGM. (a) Three-dimensional plot of the computed solution, (b) corresponding contour plot.

Figure 6 illustrates how the number of iterations and the computational time vary with the number of internal points per side, n . Discretising with n points per side results in a linear system of dimension n^2 , and each CGM iteration is dominated by a matrix-vector multiplication meaning the computational time scales approximately with the square of the system dimension. Consequently, the overall computational effort grows roughly as

$$\text{time} \sim (n^2)^2 = n^4.$$

This scaling shows that using higher spatial resolutions to improve the accuracy of the approximation significantly increases computational cost, a trend that is clearly reflected in Figure 6.

Progression of Iteration Requirement and Computational Time until Convergence with dimension for 2-D Poisson problem

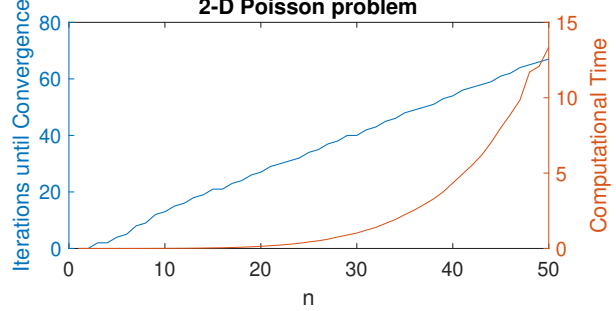


Figure 6: Number of iterations and computational time required to solve system (14) with the CGM, shown as a function of the number of interior points per side, n .

5 Condition Number

In the CGM framework, the dimension of the system n and the prescribed tolerance ε are two intuitive factors that influence the number of iterations required to reach sufficient convergence. However, systems of equal size and identical tolerances can display markedly different convergence rates. This discrepancy is governed by a property of the system matrix A known as the *condition number*.

For problems formulated with the 2-norm, the condition number is defined as

$$\kappa(A) = \|A\|_2 \|A^{-1}\|_2,$$

where the matrix norm is given by

$$\|A\|_2 = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_2}{\|\mathbf{x}\|_2}.$$

If A is symmetric and positive definite (SPD), this simplifies considerably. In this case, the 2-norm of the matrix is governed by its extremal eigenvalues, and we have

$$\kappa(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)},$$

where $\lambda_{\max}$ and $\lambda_{\min}$ denote the largest and smallest eigenvalues, respectively. Since all eigenvalues of an SPD matrix are positive, the condition number is always at least one, and it becomes large when the eigenvalues are spread over a wide range.

The condition number characterises the sensitivity of the linear system $A\mathbf{x} = \mathbf{b}$ to perturbations in either the matrix A or the right-hand side $\mathbf{b}$. A large condition number implies that small perturbations in the input data can cause disproportionately large changes in the solution $\mathbf{x}$.

For the CGM, there is a direct link between the condition number and the rate of convergence. One key result is a bound on the relative residual after k iterations:

$$\frac{\|\mathbf{r}_k\|_2}{\|\mathbf{r}_0\|_2} \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^k. \quad (14)$$

This inequality shows that a larger condition number slows down convergence, as the factor on the right-hand side approaches one when $\kappa(A)$ is large. Rearranging this relation for the number of iterations gives the well-known estimate

$$k = \mathcal{O} \left(\sqrt{\kappa(A)} \ln \left(\frac{1}{\varepsilon} \right) \right),$$

where ε is the prescribed *relative* residual tolerance. Thus, the number of iterations grows proportionally to the square root of the condition number.

To illustrate this effect, consider the one-dimensional Poisson system (10) modified by shifting the diagonal entries from 2 to $2 + \alpha$:

$$\begin{pmatrix} 2+\alpha & -1 & 0 & \cdots & 0 \\ -1 & 2+\alpha & -1 & \cdots & 0 \\ 0 & -1 & 2+\alpha & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & -1 \\ 0 & 0 & 0 & -1 & 2+\alpha \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_n \end{pmatrix} = h^2 \begin{pmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}. \quad (15)$$

The eigenvalues of this tridiagonal matrix are well known and can be expressed as

$$\lambda_i(A_\alpha) = 2 + \alpha - 2 \cos \left(\frac{i\pi}{n+1} \right), \quad i = 1, \dots, n.$$

The smallest and largest eigenvalues are obtained for $i = 1$ and $i = n$, using the identity $\cos \left(\frac{n\pi}{n+1} \right) = -\cos \left(\frac{\pi}{n+1} \right)$:

$$\lambda_{\min}(A_\alpha) = 2 + \alpha - 2 \cos \left(\frac{\pi}{n+1} \right), \quad \lambda_{\max}(A_\alpha) = 2 + \alpha + 2 \cos \left(\frac{\pi}{n+1} \right).$$

The condition number is therefore

$$\kappa(A_\alpha) = \frac{2 + \alpha + 2 \cos \left(\frac{\pi}{n+1} \right)}{2 + \alpha - 2 \cos \left(\frac{\pi}{n+1} \right)}. \quad (16)$$

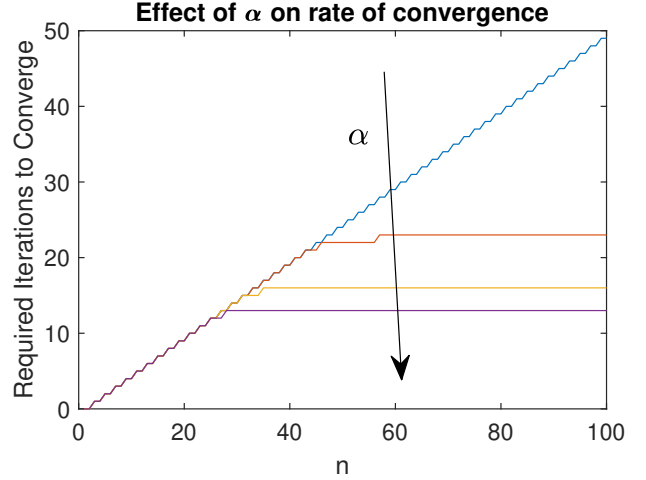


Figure 7: Convergence rates of the CGM applied to system (15) for varying α . As predicted by equation (16), the condition number decreases as α increases, resulting in faster convergence.

This expression shows that increasing α shifts the spectrum away from zero, thereby reducing the condition number and accelerating convergence. Conversely, for α close to zero (the standard Poisson case), the condition number grows approximately quadratically with n , resulting in slower convergence, as confirmed in Figure 7.

6 Preconditioning

Having established how the convergence rate of the Conjugate Gradient Method (CGM) is governed by the condition number of the system matrix, it is natural to ask: *what can be done to improve it?* The most common approach is *preconditioning*, where the original linear system is transformed in a way that preserves the exact solution but reduces the condition number. A lower condition number leads to faster convergence of the CGM.

The key idea is to select a matrix M that approximates A and is inexpensive to invert. If $M \approx A$, then the transformed matrices $M^{-1}A$, AM^{-1} , or $M^{-\frac{1}{2}}AM^{-\frac{1}{2}}$ (which approximate the identity matrix) tend to cluster their eigenvalues near 1 (though this is not guaranteed). This clustering reduces the spread of eigenvalues and, consequently, the condition number. To preserve the exact solution, M must be invertible. For certain approaches, it is also crucial that M is symmetric and positive definite (SPD).

There are three main types of preconditioning commonly employed:

Left Preconditioning The system is multiplied on the left by M^{-1} :

$$M^{-1}A\mathbf{x} = M^{-1}\mathbf{b}.$$

This form preserves the exact solution but does not guarantee that $M^{-1}A$ is SPD, even when M and A are SPD. For this reason, left preconditioning is rarely used with the CGM.

Right Preconditioning Here, the variable substitution $\mathbf{x} = M^{-1}\mathbf{y}$ is used:

$$AM^{-1}\mathbf{y} = \mathbf{b}.$$

The system is solved for $\mathbf{y}$, and then $\mathbf{x}$ is recovered. This transformation also preserves the exact solution provided M is invertible, but, as with left preconditioning, SPD is not guaranteed for AM^{-1} .

Split (or Symmetric) Preconditioning This approach combines both left and right preconditioning. Define $\mathbf{x} = M^{-\frac{1}{2}}\mathbf{z}$ and left-multiply by $M^{-\frac{1}{2}}$:

$$M^{-\frac{1}{2}}AM^{-\frac{1}{2}}\mathbf{z} = M^{-\frac{1}{2}}\mathbf{b}.$$

If M is SPD, then its symmetric square root $M^{\frac{1}{2}}$ exists and is invertible, and the resulting matrix $M^{-\frac{1}{2}}AM^{-\frac{1}{2}}$ is also guaranteed to be SPD provided A is SPD. This property makes split preconditioning the most suitable for use with the CGM.

The choice of preconditioner depends strongly on the properties of A , the scale of the problem, and the magnitude of the condition number. Methods that provide high quality preconditioning are usually more computationally expensive to employ so a natural tradeoff exists between the start up cost of preconditioning and running the CGM on a preconditioned problem. For smaller problems or for moderate condition numbers ($\approx 10^4$), Jacobi preconditioning (essentially takes the diagonal entries of A), which is negligible to compute but provides a modest improvement in the condition number, may become suitable. For larger problems (like those from discretised PDEs) or condition numbers above 10^6 , preconditioners such as multigrid (a linear system solver itself but can also be used as a preconditioner), which is computationally expensive to implement but greatly reduces the condition number, are more suitable.

Up to this point, we have not considered the role of round-off errors. Each iteration of the CGM accumulates rounding errors, which can gradually destroy the conjugacy of the search directions

and cause the computed residual to deviate from the true residual. As a result, the theoretical guarantee of convergence in at most n iterations can break down. This effect is particularly pronounced in systems where the entries of A span several orders of magnitude, amplifying the rounding effects. Scaling the system prior to iteration can mitigate this issue. Because round-off errors grow with the number of iterations, systems with a large condition number (and hence slower convergence) are more susceptible:

$$\text{Round-off error} \propto \sqrt{\kappa(A)}.$$

Thus, preconditioning not only accelerates convergence but also reduces the accumulation of numerical error.

In practice, a typical preconditioned CGM workflow consists of:

1. Estimating the condition number of the original system.
2. Selecting an appropriate preconditioner based on the estimated condition number and problem characteristics.
3. Applying the chosen preconditioning strategy and solving the preconditioned system using the CGM.
4. Recovering the solution to the original system from the preconditioned solution, if required.

Direct computation of eigenvalues to obtain the condition number (as in system (14)) is rarely feasible and approximating the eigenvalues is often more computationally expensive than running the CGM in the first place. Instead, the condition number is approximated by running the CGM on the original system and monitoring the rate of convergence. Estimating the condition number in this way streamlines the entire implementation whilst remaining computationally stable.

7 Conclusion

(The header files used to generate the results are given in Appendix 8.7, 8.8 and 8.9. The code producing the various matrices as well as the implementation is given in Appendix 8.10.)

The conjugate gradient method (CGM) has been demonstrated as a highly effective iterative technique for solving sparse, symmetric positive definite (SPD) linear systems. Its theoretical guarantee of exact convergence in at most n iterations, combined with its tendency to converge sufficiently well in far fewer iterations, makes it a practical choice for many large-scale problems. Because the method is exact in theory, it does not suffer from truncation error, and round-off errors only become significant when the system is ill-conditioned. This gives the CGM a strong degree of numerical stability, particularly for problems arising from partial differential equation (PDE) discretisations, where sparse SPD matrices occur naturally.

Preconditioning has been shown to play a central role in making the CGM practical for a wider class of problems. By transforming the linear system into one with a lower condition number, preconditioners can dramatically reduce the number of iterations required for convergence. Basic preconditioners like the Jacobi preconditioner work well when the scale of the problem or the condition number are modest. This is because it is cheap to implement and only a small reduction in condition number is required. On the other hand, multigrid preconditioners are preferred if the problem dimension or condition number are large. This is because their high computational cost is justified due to the need to reduce the condition number significantly. Estimating the condition number through the observed convergence behaviour of the CGM on the unpreconditioned system provides an efficient and computationally stable way to guide the choice of preconditioner.

While the CGM is particularly well-suited to sparse SPD systems, its use outside this context remains limited. If the original problem does not satisfy the SPD requirements, it can often be transformed accordingly; however, such transformations typically increase the condition number, sometimes significantly, which can offset the benefits of using CGM. In these situations, alternative

approaches such as direct solvers for smaller problems or GMRES for non-symmetric systems are often more efficient.

In conclusion, the CGM combines a favourable theoretical foundation with practical efficiency for a wide range of problems, particularly those involving large-scale PDE discretisations. Its effectiveness is highly dependent on the problem's conditioning and the choice of preconditioning strategy, making these factors central to its successful implementation in real-world applications.

8 Appendix

8.1

Theorem 1. *Let $\{\mathbf{e}_i\}_{i=0}^{n-1}$ be a set of A -conjugate basis vectors used as search directions to solve the linear system*

$$A\mathbf{x} = \mathbf{b},$$

where the adaptive step size is chosen according to equation (2). If the conjugate basis vectors are applied sequentially, $\mathbf{e}_0, \dots, \mathbf{e}_{n-1}$, then

$$\mathbf{e}_j^\top A\mathbf{x}_k = \mathbf{e}_j^\top \mathbf{b} \quad \text{for all } j < k.$$

Proof. We proceed by induction on k .

Base case ($k = 1$). The first update uses $\mathbf{e}_0$ as the search direction. By the definition of the optimal step size α_0 , the directional derivative of $f(\mathbf{x})$ in the $\mathbf{e}_0$ -direction vanishes at $\mathbf{x}_1$:

$$D_{\mathbf{e}_0} f(\mathbf{x}_1) = \mathbf{e}_0^\top \nabla f(\mathbf{x}_1) = \mathbf{e}_0^\top (A\mathbf{x}_1 - \mathbf{b}) = 0.$$

Hence

$$\mathbf{e}_0^\top A\mathbf{x}_1 = \mathbf{e}_0^\top \mathbf{b},$$

and the base case holds.

Induction hypothesis. Assume that for some $k \geq 1$,

$$\mathbf{e}_j^\top A\mathbf{x}_k = \mathbf{e}_j^\top \mathbf{b} \quad \text{for all } j < k.$$

Inductive step. The (k) -th update uses $\mathbf{e}_k$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{e}_k.$$

By the same optimality condition as before,

$$\mathbf{e}_k^\top (A\mathbf{x}_{k+1} - \mathbf{b}) = 0 \implies \mathbf{e}_k^\top A\mathbf{x}_{k+1} = \mathbf{e}_k^\top \mathbf{b}.$$

Now fix any $j < k$. Using linearity and the conjugacy property $\mathbf{e}_j^\top A\mathbf{e}_k = 0$,

$$\mathbf{e}_j^\top A\mathbf{x}_{k+1} = \mathbf{e}_j^\top A(\mathbf{x}_k + \alpha_k \mathbf{e}_k) = \mathbf{e}_j^\top A\mathbf{x}_k + \alpha_k \mathbf{e}_j^\top A\mathbf{e}_k = \mathbf{e}_j^\top A\mathbf{x}_k.$$

By the induction hypothesis, this equals $\mathbf{e}_j^\top \mathbf{b}$.

Therefore, the property holds for all $j < k + 1$, completing the induction. $\square$

8.2

Theorem 2. *Given $\mathbf{p}_0 \in \mathbb{R}^n$, for $i \geq 1$,*

$$\mathbf{p}_i = \mathbf{r}_i - \sum_{j < i} \frac{\mathbf{p}_j^\top A\mathbf{r}_i}{\mathbf{p}_j^\top A\mathbf{p}_j} \mathbf{p}_j.$$

generates mutually conjugate vectors.

Proof. We will proceed by induction on i . Before we proceed, note that $\mathbf{x}^\top A \mathbf{y} = \mathbf{y}^\top A \mathbf{x}$ since A is symmetric.

Base case: $i = 1$.

$$\begin{aligned} \mathbf{p}_0^\top A \mathbf{p}_1 &= \mathbf{p}_0^\top A \left(\mathbf{r}_1 - \frac{\mathbf{p}_0^\top A \mathbf{r}_1}{\mathbf{p}_0^\top A \mathbf{p}_0} \mathbf{p}_0 \right) = \mathbf{p}_0^\top A \mathbf{r}_1 - \frac{\mathbf{p}_0^\top A \mathbf{r}_1}{\mathbf{p}_0^\top A \mathbf{p}_0} \mathbf{p}_0^\top A \mathbf{p}_0 \\ &= \mathbf{p}_0^\top A \mathbf{r}_1 - \mathbf{p}_0^\top A \mathbf{r}_1 = 0. \end{aligned}$$

Thus, the base case holds.

Inductive Hypothesis A set of i mutually conjugate vectors exist. Namely, for $0 \leq j < k < i$, $\mathbf{p}_j^\top A \mathbf{p}_i = 0$.

Inductive step. Let $k < i$ be given.

$$\mathbf{p}_k^\top A \mathbf{p}_i = \mathbf{p}_k^\top A \left(\mathbf{r}_i - \sum_{j < i} \frac{\mathbf{p}_j^\top A \mathbf{r}_i}{\mathbf{p}_j^\top A \mathbf{p}_j} \mathbf{p}_j \right) = \mathbf{p}_k^\top A \mathbf{r}_i - \sum_{j < i} \frac{\mathbf{p}_j^\top A \mathbf{r}_i}{\mathbf{p}_j^\top A \mathbf{p}_j} \mathbf{p}_k^\top A \mathbf{p}_j.$$

Since the first i vectors are mutually conjugate,

$$\mathbf{p}_k^\top A \mathbf{p}_i = \mathbf{p}_k^\top A \mathbf{r}_i - \frac{\mathbf{p}_k^\top A \mathbf{r}_i}{\mathbf{p}_k^\top A \mathbf{p}_k} \mathbf{p}_k^\top A \mathbf{p}_k = 0.$$

□

8.3

Theorem 3. for $i \geq k \geq 0$, $\mathbf{r}_i^\top \mathbf{r}_k = \mathbf{r}_i^\top \mathbf{p}_k$.

Proof.

$$\begin{aligned} \mathbf{r}_i^\top \mathbf{p}_k &= \mathbf{r}_i^\top \left(\mathbf{r}_k - \sum_{j < k} \frac{\mathbf{p}_j^\top A \mathbf{r}_k}{\mathbf{p}_j^\top A \mathbf{p}_j} \mathbf{p}_j \right) = \mathbf{r}_i^\top \mathbf{r}_k - \sum_{j < k} \frac{\mathbf{p}_j^\top A \mathbf{r}_k}{\mathbf{p}_j^\top A \mathbf{p}_j} \mathbf{r}_i^\top \mathbf{p}_j = \mathbf{r}_i^\top \mathbf{r}_k - \sum_{j < k} \frac{\mathbf{p}_j^\top A \mathbf{r}_k}{\mathbf{p}_j^\top A \mathbf{p}_j} (\mathbf{b} - A \mathbf{x}_i)^\top \mathbf{p}_j \\ &= \mathbf{r}_i^\top \mathbf{r}_k - \sum_{j < i} \frac{\mathbf{p}_j^\top A \mathbf{r}_i}{\mathbf{p}_j^\top A \mathbf{p}_j} (\mathbf{p}_j^\top \mathbf{b} - \mathbf{p}_j^\top A \mathbf{x}_i). \end{aligned}$$

But $\mathbf{p}_j^\top \mathbf{b} = \mathbf{p}_j^\top A \mathbf{x}_i$ for $j < i$ by Theorem 1, so,

$$\mathbf{r}_i^\top \mathbf{r}_k = \mathbf{r}_i^\top \mathbf{p}_k$$

□

8.4

Theorem 4. for $i \neq j$, $\mathbf{r}_i^\top \mathbf{r}_j = 0$.

Proof. without loss of generality (W.L.G) let $i > j$. Then, $\mathbf{r}_i^\top \mathbf{r}_j = \mathbf{r}_i^\top \mathbf{p}_j$ by Theorem 3. Then,

$$\mathbf{r}_i^\top \mathbf{r}_j = \mathbf{p}_j^\top (\mathbf{b} - A \mathbf{x}_i) = \mathbf{p}_j^\top \mathbf{b} - \mathbf{p}_j^\top A \mathbf{x}_i.$$

But, $\mathbf{p}_j^\top \mathbf{b} - \mathbf{p}_j^\top A \mathbf{x}_i$ by Theorem 1 so $\mathbf{r}_i^\top \mathbf{r}_j = 0$.

□

8.5

Theorem 5. for $j < i$, $\mathbf{p}_j^\top \mathbf{A} \mathbf{r}_{i+1} = 0$ and $\mathbf{p}_i^\top \mathbf{A} \mathbf{r}_{i+1} = -\frac{\mathbf{r}_{i+1}^\top \mathbf{r}_{i+1}}{\alpha_i}$

Proof. let $j \leq i$ be given.

$$\mathbf{r}_{j+1} = \mathbf{r}_j - \alpha_j \mathbf{A} \mathbf{p}_j \implies \mathbf{A} \mathbf{p}_j = \frac{\mathbf{r}_j - \mathbf{r}_{j+1}}{\alpha_j}.$$

Then,

$$\mathbf{p}_j^\top \mathbf{A} \mathbf{r}_{i+1} = \mathbf{p}_j^\top \mathbf{A}^\top \mathbf{r}_{i+1} = (\mathbf{A} \mathbf{p}_j)^\top \mathbf{r}_{i+1} = \frac{(\mathbf{r}_j - \mathbf{r}_{j+1})^\top}{\alpha_j} \mathbf{r}_{i+1} = \frac{\mathbf{r}_j^\top \mathbf{r}_{i+1} - \mathbf{r}_{j+1}^\top \mathbf{r}_{i+1}}{\alpha_j}.$$

By Theorem 4, this is only non zero when $i = j$ in which case $\mathbf{p}_i^\top \mathbf{A} \mathbf{r}_{i+1} = -\frac{\mathbf{r}_{i+1}^\top \mathbf{r}_{i+1}}{\alpha_i}$. □

8.6

Theorem 6.

$$\mathbf{p}_i = \mathbf{r}_i + \frac{\mathbf{r}_i^\top \mathbf{r}_i}{\mathbf{r}_{i-1}^\top \mathbf{r}_{i-1}} \mathbf{p}_{i-1} \quad \text{is equivalent to} \quad \mathbf{p}_i = \mathbf{r}_i - \sum_{j < i} \frac{\mathbf{p}_j^\top \mathbf{A} \mathbf{r}_i}{\mathbf{p}_j^\top \mathbf{A} \mathbf{p}_j} \mathbf{p}_j.$$

Proof. By Theorem 5, $\mathbf{p}_j^\top \mathbf{A} \mathbf{r}_i = 0$ for all $j < i - 1$ and when $j = i - 1$, $\mathbf{p}_j^\top \mathbf{A} \mathbf{r}_i = -\frac{\mathbf{r}_i^\top \mathbf{r}_i}{\alpha_{i-1}}$.

The Gram-Schmidt formulation then becomes

$$\mathbf{p}_i = \mathbf{r}_i + \frac{1}{\alpha_{i-1}} \frac{\mathbf{r}_i^\top \mathbf{r}_i}{\mathbf{p}_{i-1}^\top \mathbf{A} \mathbf{p}_{i-1}} \mathbf{p}_{i-1}.$$

Using the fact that $\mathbf{r}_i^\top \mathbf{r}_i = \mathbf{r}_i^\top \mathbf{p}_i$ from theorem 3, we can plug in our expression for α_i :

$$\mathbf{p}_i = \mathbf{r}_i + \frac{\mathbf{r}_i^\top \mathbf{r}_i}{\mathbf{r}_{i-1}^\top \mathbf{r}_{i-1}} \mathbf{p}_{i-1}$$

□

8.7

```

1  #ifndef MVECTOR_H // the 'include guard'
2  #define MVECTOR_H // see C++ Primer Sec. 2.9.2
3
4  #include <vector>
5  #include <iostream>
6  #include <algorithm>
7  #include <cmath>
8
9  // Class that represents a mathematical vector
10 class MVector
11 {
12 public:
13     // constructors
14     MVector() {}
15     explicit MVector(int n) : v(n) {}
16     MVector(int n, double x) : v(n, x) {}
17     MVector(std::initializer_list<double> l) : v(l) {}
18
19     // access element (lvalue) (see example sheet 5, q5.6)
20     double &operator[](int index)
21     {

```



```

22     return v[index];
23 }
24
25 // access element (rvalue) (see example sheet 5, q5.7)
26 double operator[](int index) const {
27     return v[index];
28 }
29
30 int size() const { return v.size(); } // number of elements
31
32 // add data to your vector
33 void push_back(double x){v.push_back(x);}
34
35 // infinity norm
36 double LInfNorm() const
37 {
38     double maxAbs = 0;
39     std::size_t s = size();
40     for (int i=0; i<s; i++)
41     {
42         maxAbs = std::max(std::abs(v[i]), maxAbs);
43     }
44     return maxAbs;
45 }
46
47 // L2 norm
48 double L2Norm() const
49 {
50     double sum = 0;
51     std::size_t s = size();
52     for (int i=0; i<s; i++)
53     {
54         sum+=v[i]*v[i];
55     }
56     return std::sqrt(sum);
57 }
58
59
60 private:
61     std::vector<double> v;
62 };
63
64 //1.2.1
65
66 inline MVector operator*(const double& lhs, const MVector& rhs)
67 {
68     MVector temp = rhs;
69     for (int i=0; i<temp.size(); i++) temp[i]*=lhs;
70     return temp;
71 }
72
73 inline MVector operator*(const MVector& rhs, const double& lhs)
74 {
75     MVector temp = rhs;
76     for (int i=0; i<temp.size(); i++) temp[i]*=lhs;
77     return temp;
78 }
79
80 inline MVector operator/(const MVector& rhs, const double& lhs)
81 {
82     MVector temp = rhs;

```

```

83     for (int i=0; i<temp.size(); i++) temp[i]/=lhs;
84     return temp;
85 }
86
87 inline MVector operator+(const MVector& lhs, const MVector& rhs)
88 {
89     if (lhs.size()!=rhs.size())
90     {
91         std::cout<<"incompatible vector size"<<std::endl;
92         return {0};
93     }
94     MVector temp = rhs;
95     for (int i=0; i<temp.size(); i++) temp[i]=lhs[i]+rhs[i];
96     return temp;
97 }
98
99 inline MVector operator-(const MVector& lhs, const MVector& rhs)
100 {
101     MVector temp = rhs;
102     for (int i=0; i<temp.size(); i++) temp[i]=lhs[i]-rhs[i];
103     return temp;
104 }
105
106 inline double dot(const MVector& lhs, const MVector& rhs)
107 {
108     if (lhs.size()!=rhs.size())
109     {
110         std::cout<<"incompatible vectors"<<std::endl;
111         return 1e6;
112     }
113     double sum=0;
114     for(int i=0;i<lhs.size();i++)
115     {
116         sum+=lhs[i]*rhs[i];
117     }
118     return sum;
119 }
120
121 std::ostream& operator<<(std::ostream& os, const MVector& v)
122 {
123     int n = v.size();
124     os << "("<<v[0];
125     for(int i=1; i<n;i++) os<<","<<v[i];
126     os << ")";
127     return os;
128 }
129
130 #endif

```

8.8

```

131 #ifndef MMATRIX_H // the 'include guard'
132 #define MMATRIX_H
133
134 #include <vector>
135
136 // Class that represents a mathematical matrix
137 class MMatrix
138 {
139 public:

```

```

140 // constructors
141 MMatrix() : nRows(0), nCols(0) {}
142 MMatrix(int n, int m, double x = 0) : nRows(n), nCols(m), A(n * m, x) {}
143
144 // set all matrix entries equal to a double
145 MMatrix &operator=(double x)
146 {
147     for (unsigned i = 0; i < nRows * nCols; i++) A[i] = x;
148     return *this;
149 }
150
151 // access element, indexed by (row, column) [rvalue]
152 double operator()(int i, int j) const
153 {
154     return A[j + i * nCols];
155 }
156
157 // access element, indexed by (row, column) [lvalue]
158 double &operator()(int i, int j)
159 {
160     return A[j + i * nCols];
161 }
162
163 // size of matrix
164 int Rows() const { return nRows; }
165 int Cols() const { return nCols; }
166
167 private:
168     unsigned int nRows, nCols;
169     std::vector<double> A;
170 };
171
172 inline MVector operator*(const MMatrix& A, const MVector& x)
173 {
174     if (x.size() != A.Cols())
175     {
176         std::cout << "incompatible operator" << std::endl;
177         return {0};
178     }
179     MVector v(A.Rows());
180     for (int i=0; i<A.Rows(); i++)
181     {
182         v[i]=0;
183         for(int j=0; j<A.Cols();j++)
184         {
185             v[i]+=A(i,j)*x[j];
186         }
187     }
188     return v;
189 }
190
191 std::ostream& operator<<(std::ostream& os, const MMatrix& A)
192 {
193     os.width(1);
194     os << "(";
195     os.width(9);
196     os.precision(9);
197     os<<A(0,0);
198     os<<",";
199
200     for(int i=0;i<A.Rows();i++)

```

```

201     {
202         for(int j=0;j<A.Cols();j++)
203         {
204             if (i==0 && j==0) continue;
205             os.width(10);
206             os.precision(10);
207             os<<A(i,j);
208             if(j<A.Cols()-1) os<<",";
209         }
210         if(i<A.Rows()-1) os<<std::endl;
211     }
212     os << ")";
213     return os;
214 }
215
216 #endif

```

8.9

```

217 #ifndef TIMING_H
218 #define TIMING_H
219
220 #ifdef _WIN32
221     #define NOMINMAX
222     #include <windows.h>
223     #include <winbase.h>
224 #endif
225
226 #if defined(__linux__) || defined(__APPLE__)
227     #include <sys/time.h>
228 #endif
229
230 // Timer() function, returns elapsed 'wall-clock' time in seconds
231 // Time zero is set the first time Timer() is called.
232 double Timer()
233 {
234     #ifdef _WIN32
235         static LARGE_INTEGER freq, li0;
236         static bool dummy1 = QueryPerformanceFrequency(&freq);
237         static bool dummy2 = QueryPerformanceCounter(&li0);
238         LARGE_INTEGER li;
239         QueryPerformanceCounter(&li);
240         return static_cast<double>(li.QuadPart-li0.QuadPart)/static_cast<double>(freq.QuadPart)
241         ;
242     #endif
243
244     #if defined(__linux__) || defined(__APPLE__)
245         static timeval tvStart;
246         static int dummy = gettimeofday(&tvStart, 0);
247         timeval tv;
248         gettimeofday(&tv, 0);
249         return double(tv.tv_sec-tvStart.tv_sec)+double(tv.tv_usec-tvStart.tv_usec)*0.000001;
250     #endif
251 }
252 #endif

```

8.10

```

253 #include <iostream>
254 #include "mvector.h"
255 #include "mmatrix.h"
256 #include <cmath>
257 #include "timing.h"
258
259 void PoiDisEqnMatrix(MMatrix &A, MVector &b, int n)
260 {
261     MMatrix C(n,n,0);
262     MVector d(n,1.0/((n+1)*(n+1)));
263     C(0,0)=2;
264     for(int i=1; i<C.Rows();i++)
265     {
266         C(i,i)=2;
267         C(i,i-1)=-1;
268         C(i-1,i)=-1;
269     }
270     A=C; b=d;
271 }
272
273 void PoiDisEqn2dMatrix(MMatrix &A, MVector &b, int n)
274 {
275     MMatrix C(n*n,n*n,0);
276     MVector d(n*n,1.0/((n+1)*(n+1)));
277     for(int i=0; i<C.Rows();i++)
278     {
279         C(i,i)=4;
280         if(i%n!=0)
281         {
282             C(i,i-1)=-1;
283             C(i-1,i)=-1;
284         }
285         if(i>=n)
286         {
287             C(i-n,i)=-1;
288             C(i,i-n)=-1;
289         }
290     }
291     A=C; b=d;
292 }
293
294 void PoiDisEqnMatrixalt(MMatrix &A, MVector &b, int n, double alpha)
295 {
296     MMatrix C(n,n,0);
297     MVector d(n,2.5);
298     C(0,0)=2+alpha;
299     for(int i=1; i<C.Rows();i++)
300     {
301         C(i,i)=2+alpha;
302         C(i,i-1)=-1;
303         C(i-1,i)=-1;
304     }
305     A=C; b=d;
306 }
307
308 MVector ConjugateGradient(MMatrix A, MVector b, MVector &x, double tolerance)
309 {
310     if (A.Cols()!=x.size() || A.Rows()!=b.size())
311     {
312         std::cout<<"incompatible arguments"<<std::endl;

```

```

313     return {0};
314 }
315 double alpha, beta;
316 MVector r=b-A*x, currentr;
317 MVector p=r;
318 for (int iter=0; iter<b.size(); iter++)
319 {
320     // ...calculate new values for x and r here...
321     alpha=dot(r,r)/dot(p,A*p);
322     x=x+alpha*p;
323     currentr=r;
324     r=r-alpha*(A*p);
325     // check if solution is accurate enough
326     if (r.L2Norm() < tolerance)
327     {
328         std::cout<<iter<<std::endl;
329         return x;
330     }
331     // ...calculate new conjugate vector p here...
332     beta=dot(r,r)/dot(currentr,currentr);
333     p=r+beta*p;
334 }
335 std::cout<<b.size()<<std::endl;
336 return x;
337 }
338
339 MVector FastConjugateGradient(MMatrix A, MVector b, MVector &x, double tolerance)
340 {
341     if (A.Cols()!=x.size() || A.Rows()!=b.size())
342     {
343         std::cout<<"incompatible arguments"<<std::endl;
344         return {0};
345     }
346     double alpha, beta;
347     MVector r=b-A*x, currentr;
348     MVector p=r;
349     for (int iter=0; iter<b.size(); iter++)
350     {
351         // ...calculate new values for x and r here...
352         MVector q=A*p;
353         alpha=dot(r,r)/dot(p,q);
354         x=x+alpha*p;
355         currentr=r;
356         r=r-alpha*(q);
357         // check if solution is accurate enough
358         if (r.L2Norm() < tolerance)
359         {
360             std::cout<<iter<<std::endl;
361             return x;
362         }
363         // ...calculate new conjugate vector p here...
364         beta=dot(r,r)/dot(currentr,currentr);
365         p=r+beta*p;
366     }
367     std::cout<<b.size()<<std::endl;
368     return x;
369 }
370
371 MVector Steepestdescent(MMatrix A, MVector b, MVector &x, double tolerance)
372 {
373     if (A.Cols()!=x.size() || A.Rows()!=b.size())

```

```

374     {
375         std::cout<<"incompatible arguments"<<std::endl;
376         return {0};
377     }
378     double alpha, beta;
379     MVector r=b-A*x, currentr;
380     MVector p=r;
381     for (int iter=0; iter<100; iter++)
382     {
383         std::cout<<(2*p)/std::sqrt(p[0]*p[0]+p[1]*p[1])<<std::endl;
384         // ...calculate new values for x and r here...
385         alpha=dot(r,p)/dot(p,A*p);
386         x=x+alpha*p;
387         currentr=r;
388         r=r-alpha*(A*p);
389         // check if solution is accurate enough
390         if (r.L2Norm() < tolerance)
391         {
392             std::cout<<iter<<std::endl;
393             return x;
394         }
395         // ...calculate new conjugate vector p here...
396         beta=dot(r,r)/dot(currentr,currentr);
397         p=r;
398     }
399     return x;
400 }
401
402
403 int main()
404 {
405     /*
406     // ...initialise here...
407     double alpha, beta, maxIterations = 25, tolerance = 1e-6;
408     MMatrix A;
409     MVector b, x(maxIterations,0);
410     PoiDisEqnMatrix(A, b, maxIterations);
411     MVector r=b-A*x, currentr;
412     MVector p=r;
413
414     for (int iter=0; iter<maxIterations; iter++)
415     {
416         // ...calculate new values for x and r here...
417         alpha=dot(r,r)/dot(p,A*p);
418         x=x+alpha*p;
419         currentr=r;
420         r=r-alpha*(A*p);
421         // check if solution is accurate enough
422         if (r.L2Norm() < tolerance)
423         {
424             std::cout<<iter<<std::endl;
425             break;
426         }
427         // ...calculate new conjugate vector p here...
428         beta=dot(r,r)/dot(currentr,currentr);
429         p=r+beta*p;
430     }
431     std::cout<<r.L2Norm()<<std::endl;
432     std::cout<<x<<std::endl;
433     */
434

```

```

435  /*
436  int n=25;
437  MMatrix A;
438  MVector b, x(n,0);
439  PoiDisEqnMatrix(A, b, n);
440  std::cout<<ConjugateGradient(A,b,x,1e-6)<<std::endl;
441  */
442
443  /*
444  int n=25;
445  MMatrix A(2,2,0);
446  A(0,0)=2; A(1,1)=2; A(0,1)=1; A(1,0)=1;
447  MVector b={9,6}, x={1,1};
448  std::cout<<ConjugateGradient(A,b,x,1e-6)<<std::endl;
449  */
450
451  /*
452  MMatrix A(2,2,0);
453  A(0,0)=2; A(1,1)=2; A(0,1)=1; A(1,0)=1;
454  MVector b={9,6}, x={1,1};
455  double alpha, beta;
456  MVector r=b-A*x, currentr;
457  MVector p={0,1};
458  std::cout<<A*x<<std::endl;
459  std::cout<<(2*p)/std::sqrt(p[0]*p[0]+p[1]*p[1])<<std::endl;
460  std::cout<<x<<std::endl;
461  // ...calculate new values for x and r here...
462  alpha=dot(r,p)/dot(p,A*p);
463  x=x+alpha*p;
464  currentr=r;
465  r=r-alpha*(A*p);
466  // ...calculate new conjugate vector p here...
467  beta=dot(r,r)/dot(currentr,currentr);
468  p={2,-1};
469  std::cout<<A*x<<std::endl;
470  std::cout<<(2*p)/std::sqrt(p[0]*p[0]+p[1]*p[1])<<std::endl;
471  std::cout<<x<<std::endl;
472  // ...calculate new values for x and r here...
473  alpha=dot(r,p)/dot(p,A*p);
474  x=x+alpha*p;
475  currentr=r;
476  r=r-alpha*(A*p);
477  // ...calculate new conjugate vector p here...
478  beta=dot(r,r)/dot(currentr,currentr);
479  */
480
481  /*gradient descent
482  {
483  MMatrix A(2,2,0);
484  A(0,0)=2; A(1,1)=2; A(0,1)=1; A(1,0)=1;
485  MVector b={9,6}, x={1,1};
486  Steepestdescent(A, b, x, 1e-2);
487  }
488  */
489
490  /*investigating x_i
491  {
492  int n=25;
493  MMatrix A;
494  MVector b, x(n,0);
495  PoiDisEqnMatrix(A, b, n);

```



```

496         std::cout<<ConjugateGradient(A,b,x,1e-6)<<std::endl;
497     }
498     */
499
500     /*investigating iteration requirements
501     {
502         for(int n=1; n<101; n++)
503         {
504             MMatrix A;
505             MVector b, x(n,0);
506             PoiDisEqnMatrix(A, b, n);
507             double start=Timer();
508             ConjugateGradient(A,b,x,1e-6);
509             double end=Timer();
510             std::cout<<end-start<<std::endl;
511         }
512     }
513     */
514
515     /*effect of alpha
516     {
517         for(double alpha=0; alpha<2; alpha+=0.5)
518         {
519             for(int n=1; n<101; n++)
520             {
521                 MMatrix A;
522                 MVector b, x(n,0);
523                 PoiDisEqnMatrixalt(A, b, n,alpha);
524                 ConjugateGradient(A,b,x,1e-6);
525             }
526         }
527     }
528     */
529
530     /*Poisson 2d fast
531     {
532         int n=50;
533         for(int n=1;n<51;n++)
534         {
535             MMatrix A;
536             MVector b,x(n*n,0);
537             PoiDisEqn2dMatrix(A,b,n);
538             double start=Timer();
539             FastConjugateGradient(A,b,x,1e-6);
540             double end=Timer();
541             std::cout<<end-start<<std::endl;
542         }
543     }
544     */
545
546     {
547         int n=60;
548         MMatrix A;
549         MVector b, x(n*n,0);
550         PoiDisEqn2dMatrix(A, b, n);
551         std::cout<<FastConjugateGradient(A,b,x,1e-6)<<std::endl;
552     }
553     return 0;
554 }

```
